

FlashAttention: From IO-Awareness to Hardware Specialization

A Comparative Literature Review

by

Xuanning Zhou, 225045030

Final Report

Prepared for the Operating Systems course

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

This thesis presents a comparative literature review of the FlashAttention family, including FlashAttention, FlashAttention-2, and FlashAttention-3. The review studies the three papers as a continuous systems trajectory for efficient exact self-attention rather than as three independent kernel optimizations. The central argument is that the series progresses from IO-aware attention, to parallelism-aware attention, and finally to hardware-specialized asynchronous and low-precision attention.

The thesis first introduces the research background, existing problems, problem statement, connection to Operating Systems, and thesis contributions. It then reviews the necessary preliminaries: standard Transformer attention, GPU memory hierarchy, the CUDA execution model, related long-context approaches, and evaluation methodology. The three main technical chapters analyze FlashAttention, FlashAttention-2, and FlashAttention-3 separately. Each chapter discusses motivation, method, experimental evaluation, and summary, with attention to how the paper changes the dominant performance bottleneck.

Across the three papers, FlashAttention reduces HBM traffic through tiling, online softmax, recomputation, and kernel fusion. FlashAttention-2 preserves this exact-attention foundation while improving occupancy, reducing non-matmul overhead, and redesigning thread-block and warp-level work partitioning. FlashAttention-3 adapts the same exact-attention objective to NVIDIA Hopper GPUs through producer-consumer warp specialization, WGMMMA-softmax overlap, TMA-based data movement, and FP8-aware numerical design.

The thesis concludes that the FlashAttention series is best understood as a hardware-software co-design case study. Its lessons are closely connected to Operating Systems con-

cepts such as locality, scheduling, synchronization, pipelining, resource allocation, and end-to-end performance evaluation. These lessons provide a foundation for future work on distributed long-context attention, compiler support, low-precision reliability, and integration with emerging sequence-model architectures.

Acknowledgments

This draft is based on the published FlashAttention papers and their publicly available technical materials. The final acknowledgement section should be revised by the author before submission to include the actual supervisor, committee members, collaborators, and personal acknowledgements.

Contents

Abstract	i
1 Introduction	1
1.1 Research Background	1
1.2 Existing Problems	2
1.3 Problem Statement	3
1.4 Connection to Operating Systems	4
1.5 Thesis Contributions	5
1.6 Thesis Organization	6
2 Background and Preliminaries	8
2.1 Transformer Self-Attention	8
2.2 Memory Hierarchy and IO Complexity	9
2.3 GPU Execution Model	10
2.4 Related Long-Context Approaches	10
2.5 Evaluation Metrics and Methodology	11
3 FlashAttention: IO-Aware Exact Attention	14
3.1 Motivation	14
3.2 Method	15
3.3 Experimental Evaluation	16
3.4 Summary	19
4 FlashAttention-2: Better Parallelism and Work Partitioning	20
4.1 Motivation	20
4.2 Method	21
4.3 Experimental Evaluation	22
4.4 Summary	24
5 FlashAttention-3: Asynchrony and Low-Precision Co-Design	25
5.1 Motivation	25
5.2 Method	26
5.3 Experimental Evaluation	27
5.4 Summary	29

6	Synthesis and Comparative Discussion	31
6.1	A Three-Stage Technical Trajectory	31
6.2	Comparison by Systems Dimension	31
6.3	Experimental Patterns Across Papers	33
6.4	Connection to Operating Systems Concepts	34
6.5	Limitations and Open Questions	34
6.6	Summary	35
7	Conclusion and Future Work	36
7.1	Conclusion	36
7.2	Future Work	37
7.3	Final Remarks	38
A	Review Protocol	39
A.1	Paper Selection	39
A.2	Extraction Fields	40
A.3	Scope Boundaries	40
B	Supplementary Attention Derivation	41
B.1	Standard Attention	41
B.2	Online Softmax	41
B.3	IO Accounting	42
C	Reproducibility Checklist	44
C.1	Benchmark Settings	44
C.2	Correctness Checks	45
C.3	Performance Reporting	46
D	Supplementary Systems Mapping	47
D.1	Operating-Systems Concepts	47
D.2	Limitations	47

List of Figures

2.1	Blockwise view of attention used by the FlashAttention family. The central idea is to compute exact attention without materializing the full attention matrix in HBM.	11
3.1	Microbenchmark evidence for IO-aware attention. The figure compares the behavior of standard attention and FlashAttention under different memory settings, illustrating the role of HBM traffic.	16
3.2	Runtime and memory comparison between FlashAttention and multiple attention implementations. The key result is that FlashAttention keeps memory nearly linear while remaining exact.	17
3.3	Speedup over standard PyTorch attention at different sequence lengths on A100.	18
3.4	Validation perplexity curve for GPT-2 training with FlashAttention. The experiment illustrates that the kernel improvement can translate into a model-level training benefit rather than only a microbenchmark result.	18
4.1	Thread-block parallelism in FlashAttention-2. The forward pass uses independent query blocks, while the backward pass introduces additional parallelism over key-value blocks.	21
4.2	Warp-level work partitioning before and after FA2. The revised partition reduces shared-memory communication by assigning query slices to warps.	22
4.3	Forward and backward attention speed on A100. The plots report representative FA2 configurations.	23
5.1	Ping-pong scheduling used in FlashAttention-3 to overlap softmax and matrix multiplication across warpgroups.	27
5.2	Pipeline designs in FlashAttention-3. The two-stage schedule overlaps WG-MMA and softmax, while the three-stage version further deepens the pipeline.	27
5.3	FlashAttention-3 FP16/BF16 forward speed on H100 for representative causal and non-causal settings.	28

5.4	FlashAttention-3 FP8 forward speed on H100. FP8 increases throughput but requires explicit numerical design.	28
-----	--	----

List of Tables

1.1	Thesis questions, operating-systems concepts, and chapter mapping	6
2.1	Memory-hierarchy concepts used in this thesis	9
2.2	Comparison of long-context efficiency approaches	12
2.3	Evaluation levels used throughout the thesis	13
3.1	Main bottlenecks addressed by FlashAttention	15
3.2	Algorithmic ingredients of FlashAttention	17
3.3	Representative results reported for FlashAttention	19
4.1	Main method changes in FlashAttention-2	22
4.2	End-to-end training results reported for FlashAttention-2	23
5.1	Hopper features relevant to FlashAttention-3	26
5.2	Method components in FlashAttention-3	27
5.3	Pipeline ablation results reported for FlashAttention-3	29
5.4	Representative numerical error results reported for FlashAttention-3	29
6.1	Bottleneck migration across the FlashAttention series	32
6.2	Systems-level comparison of the three papers	32
6.3	How the main experiments support each paper’s claim	33
A.1	Fields used to compare the three core papers.	40
B.1	Simplified lifetime comparison for exact attention.	42
C.1	Benchmark dimensions for a focused reproduction study.	45
D.1	Compact mapping from FlashAttention techniques to OS concepts.	47

Symbols and Acronyms

This thesis follows standard notation from the Transformer and GPU-optimization literature. Scalars are written in italic lowercase letters, vectors in bold lowercase letters, and matrices in bold uppercase letters unless stated otherwise.

Symbols

$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	query, key, and value matrices
$\mathbf{S}$	attention score matrix, typically $\mathbf{QK}^\top$
$\mathbf{P}$	row-wise softmax attention probability matrix
$\mathbf{O}$	attention output matrix
N	sequence length
d	head dimension
B_r, B_c	query and key-value tile sizes
m, ℓ, L	online softmax statistics and log-sum-exp quantities
α	scaling factor, typically $1/\sqrt{d}$
FLOPs	floating-point operations
TFLOPs/s	throughput in trillions of floating-point operations per second

Acronyms

HBM	High Bandwidth Memory
SRAM	on-chip static random-access memory
SM	Streaming Multiprocessor
CTA	Cooperative Thread Array, i.e., thread block
MHA	Multi-Head Attention
MQA	Multi-Query Attention
GQA	Grouped-Query Attention
LRA	Long Range Arena benchmark
GEMM	General Matrix Multiplication
TMA	Tensor Memory Accelerator on NVIDIA Hopper
WGMMA	Warpgroup Matrix Multiply-Accumulate instruction
FP16/BF16/FP8	commonly used floating-point formats
IO-aware	optimization guided by memory movement across the hierarchy

Chapter 1

Introduction

1.1 Research Background

Transformer models have become a dominant architecture for language modeling, vision, and many other sequence-centered tasks since the introduction of self-attention [13, 14, 34, 45]. Their success is strongly connected to the flexibility of the attention operator. Instead of processing a sequence through a fixed local receptive field, attention allows each token to condition on all other tokens. This gives the model a direct mechanism for long-range interaction and makes it possible to combine information from distant parts of an input in a single layer.

The same property also makes attention a systems bottleneck. For a sequence of length N and head dimension d , the standard attention computation forms scores $\mathbf{QK}^\top$, applies a row-wise softmax, and multiplies the result by $\mathbf{V}$. The intermediate score and probability matrices are of size $N \times N$. When N grows from thousands to tens of thousands, these intermediate tensors quickly dominate both memory usage and memory traffic. This effect is especially severe during training, where intermediate values are often retained for the backward pass.

Long context has become increasingly important. Legal documents, scientific articles, code repositories, retrieval-augmented prompts, dialogue histories, audio streams, and video tokens all place pressure on the context window. Model developers therefore face

a practical question: how can exact attention be made fast enough and memory-efficient enough to remain usable at long sequence lengths? Prior long-context work studies this pressure through benchmarks and architectures such as Transformer-XL, Long Range Arena, Longformer, and BigBird [4, 9, 41, 51]. Approximate attention methods reduce asymptotic cost by changing the operator, but they may introduce accuracy changes, model-specific assumptions, or engineering complexity [8, 21, 46]. The FlashAttention line of work takes a different path. It keeps the mathematical definition of attention intact and changes how the computation is executed on modern GPUs.

This thesis studies three papers in that line: FlashAttention [11], FlashAttention-2 [10], and FlashAttention-3 [38]. The three papers are not merely successive versions of a library routine. They reveal a broader research pattern in which algorithm design, memory hierarchy, thread scheduling, and hardware generation must be considered together. In this sense, the FlashAttention series is a useful case study for an Operating Systems course: it illustrates how performance is often determined by locality, scheduling, synchronization, resource allocation, and the cost of data movement rather than by arithmetic counts alone.

1.2 Existing Problems

The first existing problem is the gap between the mathematical expression of attention and its direct implementation. In high-level notation, attention is a short formula:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}.$$

In a conventional implementation, however, this formula may be translated into several separate kernels: one for the score matrix, one for masking, one for softmax, one for dropout, and one for the final multiplication. Each kernel reads and writes large tensors through high-bandwidth memory (HBM). Although the arithmetic work is large, the actual runtime can be dominated by memory traffic and synchronization between operations.

The second problem is that reducing memory traffic alone is not sufficient. If a kernel

avoids materializing the full attention matrix but leaves many GPU streaming multiprocessors underutilized, performance remains far from the hardware limit. In this case the bottleneck shifts from global memory traffic to occupancy, warp-level work partitioning, and non-matrix-multiplication overhead. FlashAttention-2 is motivated precisely by this observation: once FlashAttention removes the largest memory bottleneck, a second layer of parallel execution problems becomes visible.

The third problem is hardware evolution. A kernel that is well matched to one GPU generation may not fully exploit the next one. NVIDIA Hopper GPUs expose features such as the Tensor Memory Accelerator (TMA), asynchronous WGMMMA operations, and FP8 Tensor Core support [28]. These features change the cost model, and NVIDIA’s CUDA/PTX documentation makes clear that these mechanisms are exposed through lower-level programming abstractions [29, 30]. A good A100-style kernel can still leave performance on the table if it treats compute, memory movement, and softmax as a mostly synchronous pipeline. FlashAttention-3 therefore shows that exact attention optimization is not a one-time software improvement, but an ongoing hardware-software co-design problem.

The fourth problem is evaluation. Attention kernels must be judged at several levels. Kernel-level throughput, memory footprint, numerical error, and end-to-end training time are all relevant, but they answer different questions. A method may improve raw TFLOPs/s without improving training speed; another method may be fast but numerically fragile under low precision. A careful literature review must therefore connect algorithmic mechanisms to multiple experimental outcomes rather than relying on a single headline speedup.

1.3 Problem Statement

This thesis asks how the FlashAttention series progressively improves exact self-attention by reorganizing the interaction between the attention algorithm and the GPU execution substrate. More specifically, it investigates four questions:

1. How does FlashAttention convert exact attention from a memory-intensive sequence of tensor operations into an IO-aware fused kernel?

2. How does FlashAttention-2 improve parallelism, reduce non-matmul overhead, and redesign work partitioning without changing the exact attention result?
3. How does FlashAttention-3 adapt the same exact-attention objective to Hopper-specific asynchronous execution and FP8 computation?
4. What general operating-systems and computer-systems lessons can be extracted from the three papers when they are read together?

The scope of the thesis is a comparative literature review. It does not reimplement the kernels or reproduce every benchmark. Instead, it synthesizes the methods, experimental evidence, and systems implications reported in the three papers. The goal is to explain the technical progression clearly enough that the reader can see why each later paper exists, what bottleneck it addresses, and what remains unresolved.

1.4 Connection to Operating Systems

The FlashAttention papers are machine-learning systems papers, but the core ideas are deeply related to operating-systems concepts. The most direct connection is memory hierarchy. Operating systems and architecture courses emphasize that memory is not uniform: registers, caches, shared memory, DRAM, and storage have different capacities and latencies. The working-set model [12], IO-complexity analysis [1], and roofline-style performance modeling [47] all express the same principle: computation becomes efficient only when the active working set is organized around the available fast memory.

FlashAttention applies this principle to attention. Instead of storing the full attention matrix in HBM, it tiles $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ so that useful blocks fit in on-chip memory. This is analogous to blocking in matrix multiplication or cache-aware loop transformations [48]. The implementation is not faster because it changes the definition of attention; it is faster because it changes locality.

A second connection is scheduling. Operating systems schedule processes and threads over limited resources; GPU kernels schedule thread blocks and warps over streaming mul-

tiprocessors. FlashAttention-2 shows that after memory traffic is reduced, the main question becomes how to expose enough parallelism and assign work to warps so that the hardware stays busy. The paper’s sequence parallelism and warp-level partitioning can be interpreted as scheduling decisions inside a specialized runtime.

A third connection is synchronization and pipelining. OS-level systems often overlap IO with computation, pipeline producer-consumer stages, and reduce blocking synchronization. FlashAttention-3 brings this idea into a single GPU kernel by separating producer and consumer warpgroups and overlapping memory movement, WGMMMA, and softmax. The result resembles a small operating system inside the kernel: resources are partitioned, tasks are scheduled, and communication is coordinated to minimize idle time.

Finally, the FlashAttention series highlights the systems principle that an optimization should be evaluated under realistic end-to-end workloads [23]. A kernel that looks attractive in isolation must still improve full Transformer training or inference. This thesis therefore treats the three papers not only as algorithm papers, but also as case studies in resource management for modern AI workloads.

1.5 Thesis Contributions

This thesis makes four contributions.

1. It organizes FlashAttention, FlashAttention-2, and FlashAttention-3 as a single technical trajectory rather than three isolated papers.
2. It explains the progression of bottlenecks from HBM traffic, to occupancy and warp communication, to hardware-specialized asynchronous execution and low precision.
3. It connects the attention-kernel design choices to operating-systems concepts including locality, scheduling, synchronization, pipelining, and performance evaluation.
4. It compares the experimental evidence across the three papers and identifies open problems for future exact-attention systems.

Table 1.1: Thesis questions, operating-systems concepts, and chapter mapping

Chapter	Main technical question	Systems concept emphasized
Chapter 2	What background is needed to understand efficient exact attention?	Memory hierarchy, GPU execution model, evaluation metrics
Chapter 3	How does FlashAttention avoid materializing the attention matrix?	Locality, tiling, IO complexity, recomputation
Chapter 4	Why is better parallelism needed after IO traffic is reduced?	Scheduling, occupancy, warp-level work partitioning
Chapter 5	How should attention exploit Hopper hardware?	Asynchrony, pipelining, producer-consumer execution, low precision
Chapter 6	What do the three papers collectively show?	Bottleneck migration, co-design, cross-layer performance analysis

1.6 Thesis Organization

The remainder of the thesis is organized as follows. Chapter 2 introduces background and preliminaries, including standard attention, GPU memory hierarchy, the execution model of modern accelerators, related long-context methods, and evaluation methodology. Chapter 3 analyzes FlashAttention with emphasis on IO awareness, tiling, online softmax, and recomputation. Chapter 4 studies FlashAttention-2, focusing on improved parallelism, work partitioning, and the reduction of non-matmul overhead. Chapter 5 discusses FlashAttention-3 and its use of Hopper-specific asynchronous execution and low-precision design. Chapter 6 synthesizes the three papers and compares their systems implications. Chapter 7 concludes the thesis and outlines future work.

This ordering is intentional. The background chapter first fixes the notation and hard-

ware vocabulary used throughout the report, so that the later chapters can focus on the design choices made by each paper. The three paper-specific chapters then follow the historical order of the FlashAttention series. This allows the reader to see a sequence of bottlenecks rather than a disconnected set of optimizations: memory movement is addressed first, parallel execution is addressed second, and hardware-specialized asynchronous execution is addressed third.

The synthesis chapter is placed after the three technical chapters because its main purpose is comparative. It revisits the same issues from a cross-paper perspective: which bottleneck each paper identifies, which GPU resource becomes critical after the previous bottleneck is reduced, and how the reported experiments support the claimed improvement. The final chapter then separates what is already established by the literature from what remains open for future long-context systems.

Chapter 2

Background and Preliminaries

2.1 Transformer Self-Attention

Self-attention is the central primitive of the Transformer architecture [45], and it underlies widely used language and vision Transformer models [13, 14, 34]. For a single attention head, the input sequence is projected into query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$. The output is

$$\mathbf{O} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}.$$

Multi-head attention repeats this operation across several heads and combines their outputs. The operator is simple to express, but the direct implementation has two expensive intermediate tensors: the score matrix $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$ and the probability matrix $\mathbf{P} = \text{softmax}(\mathbf{S})$. Both have N^2 elements.

The forward pass is only part of the cost. During training, the backward pass requires gradients with respect to $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. A naive implementation often stores $\mathbf{P}$ or other intermediate values so that the backward pass can reuse them. This increases memory footprint and HBM traffic. Long-context training therefore becomes limited not only by compute, but also by the storage and movement of intermediate tensors [11, 33].

Masking and dropout add further complications. Causal masks are required for autoregressive language modeling, padding masks are required for variable length batches, and

dropout is often applied to attention probabilities during training [45]. These operations are relatively cheap in FLOPs, but they can be memory-bound. A high-performance attention implementation must therefore treat matrix multiplication, softmax, masking, dropout, and the backward pass as one coupled dataflow rather than as independent high-level operations.

2.2 Memory Hierarchy and IO Complexity

Modern GPUs provide enormous arithmetic throughput, but data movement is hierarchical. Registers and on-chip memory are small and fast; HBM is much larger but slower and more energy expensive. This is the same systems principle that underlies cache blocking, working-set analysis, and IO complexity [1, 12, 16, 48]. An operator that repeatedly moves large intermediates through HBM can underperform even if its arithmetic count is theoretically acceptable; this data-movement emphasis also appears in transformer-system studies [17].

The roofline model provides a useful way to reason about this effect [47]. If an operator has low arithmetic intensity, then its performance is limited by memory bandwidth rather than peak FLOPs. Standard attention contains high-throughput matrix multiplications, but the surrounding softmax and materialization steps can lower effective arithmetic intensity. FlashAttention improves this situation by increasing reuse of loaded $\mathbf{K}$ and $\mathbf{V}$ tiles and avoiding writes of the $N \times N$ intermediate matrix.

Table 2.1: Memory-hierarchy concepts used in this thesis

Concept	Relevance to attention kernels
Working set	The active query, key, value, and softmax statistics should fit in fast memory whenever possible.
Tiling	The computation is decomposed into blocks that increase reuse and reduce HBM traffic.
Arithmetic intensity	Performance depends on the ratio between useful compute and bytes moved through memory.
Recomputation	Some intermediate values are recomputed during the backward pass to reduce storage pressure.
Kernel fusion	Multiple logical operations are combined so that intermediate tensors do not travel through HBM.

2.3 GPU Execution Model

The FlashAttention series is also shaped by the GPU execution model. A CUDA kernel is organized into thread blocks, warps, registers, shared memory, and global memory [29]. Thread blocks are scheduled onto streaming multiprocessors (SMs), and warps execute in SIMD-like groups. Tensor Cores provide high throughput for matrix multiplication, while other operations such as exponentiation, normalization, masking, and data movement may become bottlenecks.

This model explains why exact attention cannot be optimized by looking only at FLOPs. Matrix multiplication can be accelerated by Tensor Cores, but softmax uses scalar operations and reductions. Shared memory can reduce HBM traffic, but excessive shared-memory communication can introduce synchronization overhead. Occupancy can improve throughput, but too many registers or too much shared memory per block can reduce the number of resident blocks. These tradeoffs are central to FlashAttention-2.

New GPU generations also change the available execution primitives. The V100 and A100 generations were studied through both vendor specifications and microbenchmarking [18, 19, 26, 27]. H100 GPUs add Hopper-specific features such as TMA, WGMMMA, and FP8 support [24, 28, 30]. FlashAttention-3 is motivated by the fact that using these features well requires a different schedule, not merely recompiling the previous kernel.

2.4 Related Long-Context Approaches

Long-context efficiency has been studied through several families of methods. Approximate attention methods reduce asymptotic cost by modifying the attention pattern or replacing the softmax kernel [42]. Sparse attention and block-sparse attention restrict which tokens attend to one another [4, 7, 36, 51]. Low-rank and projection-based methods approximate the attention matrix [46, 50]. Kernelized methods approximate softmax attention using feature maps [8, 20]. Other efficient variants combine sparse and low-rank structure or use locality-sensitive hashing [5, 21]. These approaches can reduce cost, but they also change the operator

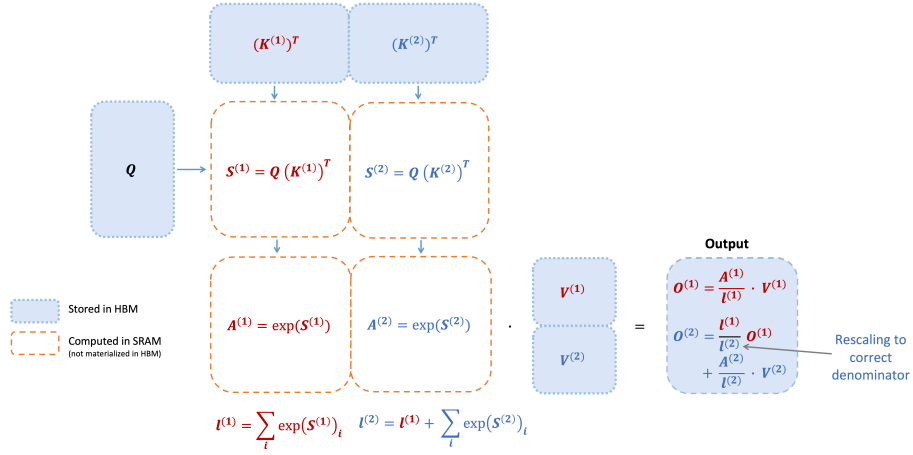


Figure 2.1: Blockwise view of attention used by the FlashAttention family. The central idea is to compute exact attention without materializing the full attention matrix in HBM. Source: [10].

or impose additional assumptions.

Another line of work avoids attention or supplements it with other sequence models. State-space and recurrent-style models reduce the need for full quadratic interaction [15]. Decoder-side variants such as multi-query and grouped-query attention reduce key-value memory movement in generation settings [2, 39]. These methods are important, but attention remains a central primitive in many high-performing systems. Exact attention optimization therefore remains valuable even when alternative architectures exist.

The FlashAttention family belongs to a third category: exact attention software optimization. Its goal is not to reduce the mathematical complexity of attention, but to reduce the actual cost of executing it. This distinction is important. The output remains identical to standard attention up to numerical rounding, so the method can be inserted into existing Transformer models with minimal algorithmic changes.

2.5 Evaluation Metrics and Methodology

The three FlashAttention papers use several evaluation levels. Kernel-level measurements report runtime, memory usage, bandwidth, and TFLOPs/s for the attention operator itself. These measurements reveal whether a design better uses the GPU. However, they do not

Table 2.2: Comparison of long-context efficiency approaches

Approach	Main idea	Main tradeoff
Sparse attention	Restrict attention to selected token pairs	Can miss useful long-range interactions if the pattern is poorly matched
Low-rank attention	Approximate the attention matrix with lower-dimensional projections	Accuracy depends on rank and input structure
Kernelized attention	Replace softmax attention with feature-map approximations	Changes numerical behavior and implementation details
Alternative sequence models	Use recurrent or state-space mechanisms	May require architectural changes and different training recipes
FlashAttention family	Preserve exact attention but change GPU dataflow	Requires careful low-level kernel engineering

fully determine model-level impact because attention is only one component of a Transformer layer.

End-to-end training measurements report throughput for GPT-style or BERT-style models. These results answer a more practical question: does the kernel improve the full training loop? Model-level evaluation is important because a speedup inside attention may be diluted by feed-forward layers, optimizer overhead, communication, or data loading; the FlashAttention papers compare against framework and system baselines such as PyTorch, HuggingFace Transformers, Megatron-LM, xFormers, and Triton [22, 32, 40, 44, 49].

Numerical evaluation becomes especially important in FlashAttention-3. FP8 can increase throughput, but it also introduces quantization error and sensitivity to outliers. The relevant question is therefore not only whether the kernel is fast, but whether it remains accurate enough for training or inference. This is why the FlashAttention-3 paper evaluates both throughput and numerical error.

Table 2.3: Evaluation levels used throughout the thesis

Level	Typical metric	Interpretation
Kernel runtime	Milliseconds per attention call	Measures direct operator speed
Memory footprint	Peak memory or asymptotic storage	Determines feasible sequence length and batch size
Hardware utilization	TFLOPs/s or percentage of peak	Indicates how well Tensor Cores and SMs are used
End-to-end training	TFLOPs/s/GPU or time-to-accuracy	Measures impact on full model training
Numerical behavior	Error relative to high-precision reference	Determines whether low precision is usable

Chapter 3

FlashAttention: IO-Aware Exact Attention

3.1 Motivation

FlashAttention begins from a simple but important observation: standard attention is often bottlenecked by memory traffic rather than by matrix multiplication alone [11]. In a direct implementation, the score matrix $\mathbf{S}$ and probability matrix $\mathbf{P}$ are written to and read from HBM. Since both matrices have N^2 elements, their cost becomes dominant when sequence length grows. This is especially problematic because the attention output has only Nd elements. The direct implementation therefore creates a large temporary working set that is not part of the final output.

The key motivation is to preserve exact attention while removing unnecessary data movement. Many efficient-attention methods reduce memory or runtime by approximating the attention pattern or matrix structure [4, 7, 8, 21, 46]. FlashAttention instead asks whether exact attention can be implemented with an IO-aware dataflow. This shift is conceptually important: it treats attention as a systems problem, where the main challenge is not changing the formula but scheduling the computation around the memory hierarchy.

The motivation can be summarized in three points. First, GPUs are very good at matrix multiplication, but the surrounding softmax, masking, and dropout operations are relatively

memory-bound. Second, writing the full attention matrix to HBM destroys locality because each block of $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ could otherwise be reused on chip. Third, training requires a backward pass, so simply optimizing the forward pass is insufficient. A useful exact-attention kernel must also control what information is saved for gradients.

Table 3.1: Main bottlenecks addressed by FlashAttention

Bottleneck	Direct implementation	FlashAttention response
Score matrix storage	Materializes $N \times N$ scores in HBM	Computes blockwise scores on chip
Softmax normalization	Requires row-wise global information	Maintains online row maxima and normalization terms
Backward memory cost	Stores large intermediate probabilities	Stores compact statistics and recomputes local blocks
Kernel boundaries	Multiple launches move tensors repeatedly	Fuses attention sub-operations into one dataflow

3.2 Method

The method is built around tiling, online softmax, and recomputation. Tiling partitions $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into blocks that fit in on-chip memory. A query block is processed against key-value blocks one at a time. Instead of forming a full row of the attention matrix, the kernel computes local score blocks, updates softmax statistics, and accumulates the partial output.

The difficulty is softmax. If a row is processed in pieces, the maximum and normalization denominator may change as new key blocks are seen. FlashAttention uses an online softmax recurrence related to earlier online normalizer techniques [25]. For each row, it maintains a running maximum m and normalization factor ℓ . When a new block of scores arrives, the old partial result is rescaled to match the new maximum before being combined with the new block. This preserves exactness while allowing the row to be processed incrementally.

The backward pass uses recomputation. Instead of storing all attention probabilities, FlashAttention stores the output and softmax normalization statistics. During the backward pass, local score blocks and probabilities are recomputed on chip as needed. This trades additional arithmetic for a large reduction in memory traffic. The tradeoff is favorable because

matrix multiplication is relatively cheap on modern GPUs compared with moving large intermediate matrices through HBM.

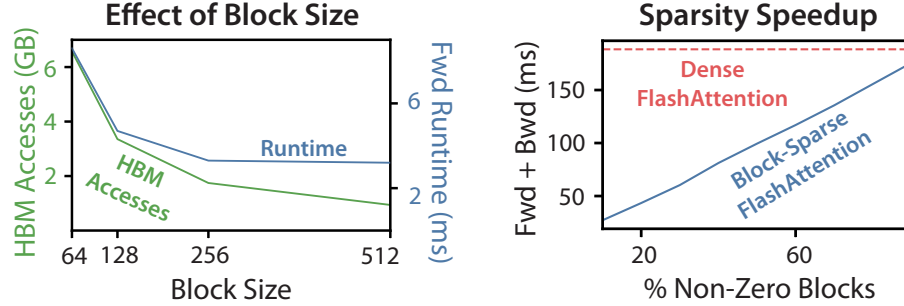


Figure 3.1: Microbenchmark evidence for IO-aware attention. The figure compares the behavior of standard attention and FlashAttention under different memory settings, illustrating the role of HBM traffic. Source: [11].

From an operating-systems perspective, the method resembles cache blocking and working-set control. The kernel intentionally chooses a working set small enough for on-chip memory [1, 48]. It also uses recomputation in the same spirit as checkpointing: some intermediate data is discarded and later regenerated to reduce memory pressure [6]. The design is therefore not only an attention trick, but also an instance of a general systems tradeoff between storage and computation.

FlashAttention also introduces a block-sparse extension. The extension is not the main focus of this thesis, but it is useful conceptually. Once the exact attention kernel is reorganized around tiles, sparse patterns can be expressed over tiles. This shows that IO-aware exact attention can become a substrate for other long-context methods rather than a replacement for them [4, 5, 7].

3.3 Experimental Evaluation

The FlashAttention paper evaluates both kernel-level performance and end-to-end model impact. At the kernel level, FlashAttention shows linear memory growth with sequence length, whereas standard exact attention requires quadratic intermediate storage. This directly validates the IO-aware design: the method does not merely improve constant factors,

Table 3.2: Algorithmic ingredients of FlashAttention

Ingredient	Mechanism	Effect
Tiling	Process query and key-value blocks in on-chip memory	Avoids full attention-matrix materialization
Online softmax	Track row maxima and normalization statistics across blocks	Preserves exact softmax despite blockwise execution
Recomputation	Recreate local attention probabilities during backward pass	Reduces stored activation memory
Kernel fusion	Combine score computation, softmax, masking, dropout, and value aggregation	Reduces repeated HBM reads and writes

but changes the memory footprint that limits long-context training.

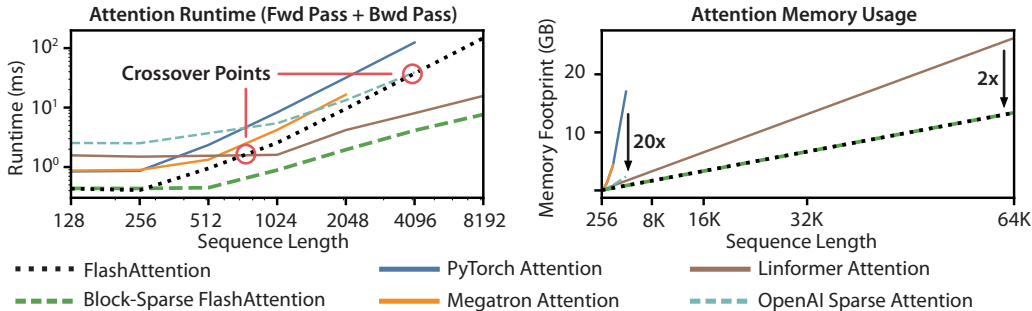


Figure 3.2: Runtime and memory comparison between FlashAttention and multiple attention implementations. The key result is that FlashAttention keeps memory nearly linear while remaining exact. Source: [11].

The paper also reports speedups across hardware platforms and sequence lengths. Figure 3.3 shows representative A100 speedup results. The important pattern is that the advantage grows as sequence length increases. This is consistent with the memory-traffic explanation: longer sequences increase the cost of materializing $\mathbf{S}$ and $\mathbf{P}$, so avoiding those tensors becomes more valuable.

At the model level, FlashAttention improves both training speed and feasible context length. The paper reports faster BERT-large training, faster GPT-2 training, and improved ability to train GPT-2 with longer context [13, 34]. These results matter because they show that the kernel-level gain survives in realistic model workloads.

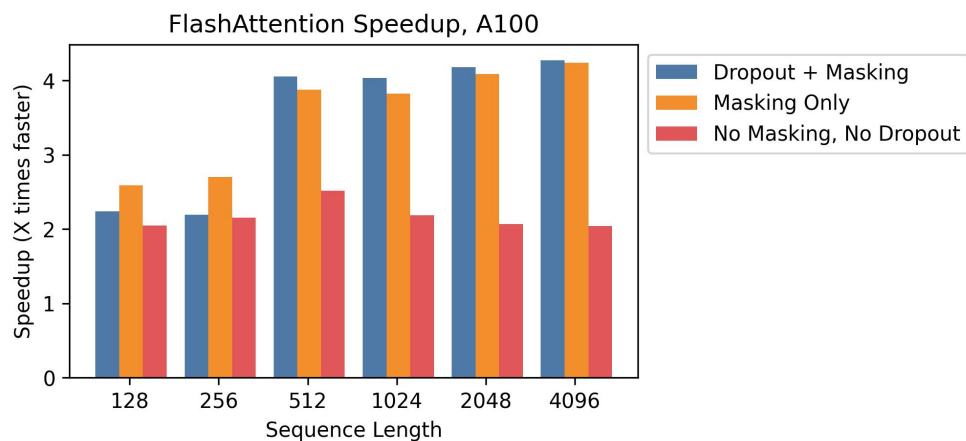


Figure 3.3: Speedup over standard PyTorch attention at different sequence lengths on A100. Source: [11].

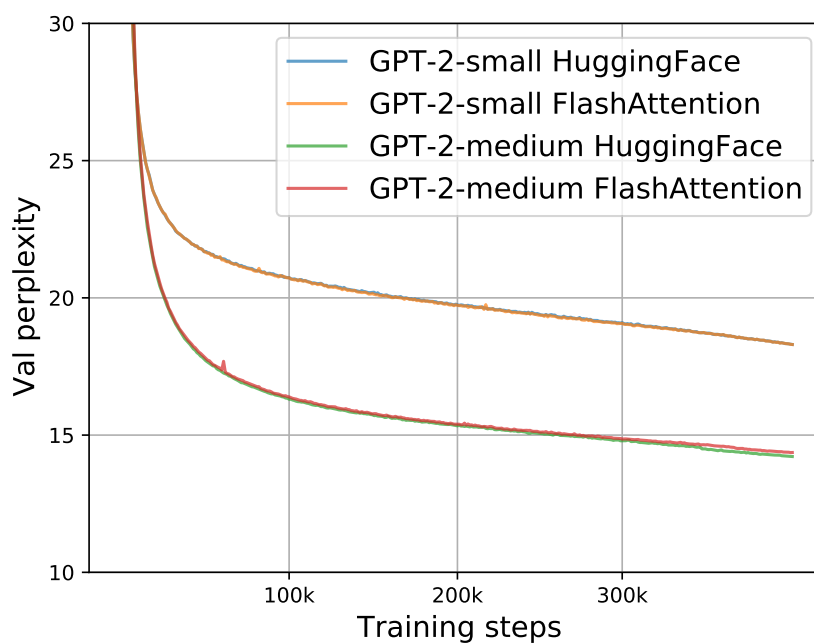


Figure 3.4: Validation perplexity curve for GPT-2 training with FlashAttention. The experiment illustrates that the kernel improvement can translate into a model-level training benefit rather than only a microbenchmark result. Source: [11].

Table 3.3: Representative results reported for FlashAttention

Evaluation target	Reported finding
BERT-large training	Training time to target accuracy is reduced relative to the baseline implementation.
GPT-2 training	Training speed improves over HuggingFace and Megatron-LM baselines [40, 49].
Kernel memory usage	Memory scales linearly with sequence length instead of storing N^2 attention probabilities.
Long Range Arena	Exact attention becomes competitive with efficient-attention variants while preserving the original operator [41].
Long-context GPT-2	Longer context training becomes feasible with improved perplexity relative to shorter-context baselines.

The evaluation also clarifies the limits of the first paper. FlashAttention is substantially faster than a standard implementation, but the achieved throughput is still below highly optimized GEMM kernels. This is not a failure; it identifies the next bottleneck. Once the most severe HBM materialization cost is removed, overhead from non-matmul operations, synchronization, and parallel work distribution becomes more visible. FlashAttention-2 is best understood as a response to this newly exposed bottleneck.

3.4 Summary

FlashAttention establishes the foundation of the series. Its central contribution is to show that exact attention can be made memory-efficient by redesigning the dataflow around the GPU memory hierarchy. The method combines tiling, online softmax, recomputation, and kernel fusion so that large intermediate matrices are never materialized in HBM. In operating-systems terms, it is a case study in locality optimization and working-set management.

The paper also sets up the remaining questions: why GEMM-like efficiency remains difficult, how work should be assigned across thread blocks and warps, and how kernels should change for asynchronous, lower-precision hardware. These questions lead naturally to FlashAttention-2 and FlashAttention-3.

Chapter 4

FlashAttention-2: Better Parallelism and Work Partitioning

4.1 Motivation

FlashAttention-2 starts from a different bottleneck than FlashAttention. FlashAttention removes the dominant HBM materialization cost, but the resulting kernel still does not reach the efficiency of optimized GEMM on A100 GPUs [10, 18]. The reason is that the remaining work contains non-matmul operations, synchronization, and work-partitioning overhead. Once IO traffic is reduced, the question becomes how to keep the GPU busy.

The paper argues that the original FlashAttention uses parallelism conservatively. It parallelizes primarily over batch size and attention heads. This is effective when there are many heads and a large batch, but long-context training often uses smaller batches because each sequence consumes more memory [10, 11]. In that setting, the sequence dimension itself must become a source of parallel work. The paper also targets warp-level communication: the original kernel may require warps to combine partial results through shared memory, which adds synchronization and memory traffic inside the thread block.

The bottleneck therefore moves in three steps: standard attention is dominated by HBM traffic from N^2 intermediates; FlashAttention removes that dominant traffic but exposes underutilization and non-matmul overhead; FlashAttention-2 responds by increasing paral-

lelism and reducing synchronization inside the kernel.

4.2 Method

FlashAttention-2 refines both the algorithmic update and the execution schedule. The first refinement reduces non-matmul FLOPs. Instead of repeatedly rescaling the output during the blockwise softmax update, the method maintains an unnormalized output accumulator and applies final scaling at the end. This reduces scalar operations that do not run on Tensor Cores. It also stores a log-sum-exp quantity that simplifies state management for the backward pass.

The second refinement increases thread-block parallelism. For the forward pass, FlashAttention-2 parallelizes over query blocks as well as batch and heads. For the backward pass, it parallelizes over key-value column blocks and coordinates gradient accumulation. The important point is that long sequences themselves become a source of parallel work rather than only a source of cost.

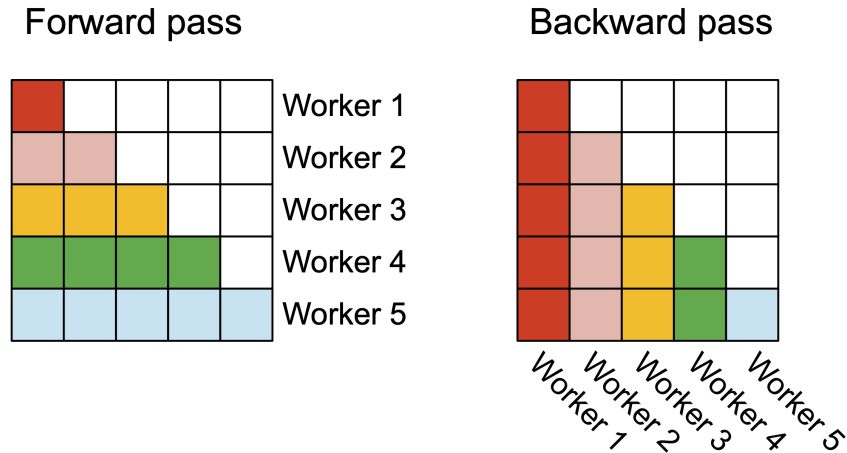


Figure 4.1: Thread-block parallelism in FlashAttention-2. The forward pass uses independent query blocks, while the backward pass introduces additional parallelism over key-value blocks. Source: [10].

The third refinement is warp-level work partitioning. The original FlashAttention design is closer to split-K partitioning, where warps process different key-value slices and then combine partial results. FlashAttention-2 instead partitions the query block across warps. Each

warp owns a different slice of the output, while key-value tiles are shared. This reduces warp-to-warp communication and makes the work more directly aligned with output generation.

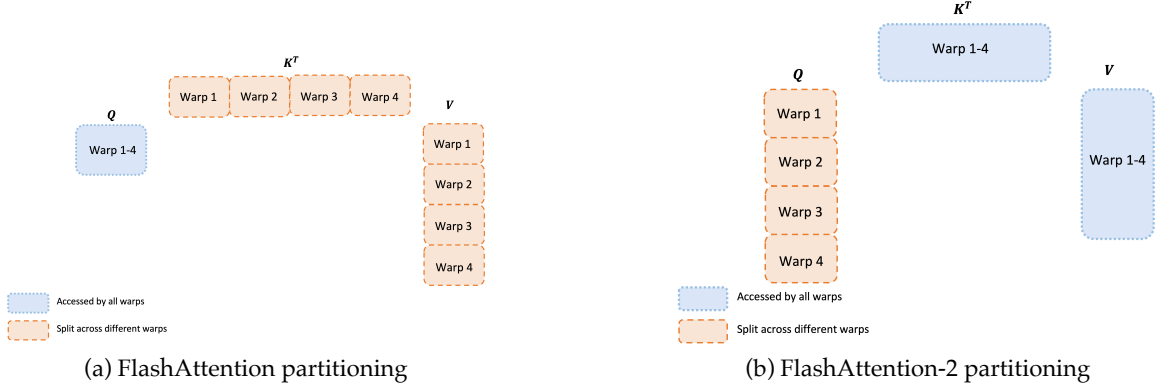


Figure 4.2: Warp-level work partitioning before and after FlashAttention-2. FlashAttention-2 reduces shared-memory communication by assigning query slices to warps. Source: [10].

From a systems perspective, this is a scheduling improvement: the mathematical work is nearly the same, but the assignment of work to execution resources is more efficient.

Table 4.1: Main method changes in FlashAttention-2

Change	Mechanism	Expected effect
Fewer non-matmul operations	Use unnormalized accumulator and final scaling	Less scalar overhead between Tensor Core operations
More block parallelism	Split work along sequence dimension	Higher occupancy for long-context settings
Query-sliced warp partitioning	Assign different query rows to different warps	Less shared-memory communication
Revised backward pass	Parallelize across column blocks with coordinated accumulation	Better backward throughput

4.3 Experimental Evaluation

The experimental results show that FlashAttention-2 improves throughput substantially over FlashAttention on A100 GPUs. The paper reports roughly $1.7\times$ to $3.0\times$ speedups relative to FlashAttention across common settings and reaches a much larger fraction

of theoretical peak throughput [10]. The improvement is especially meaningful because FlashAttention was already an optimized exact-attention kernel.

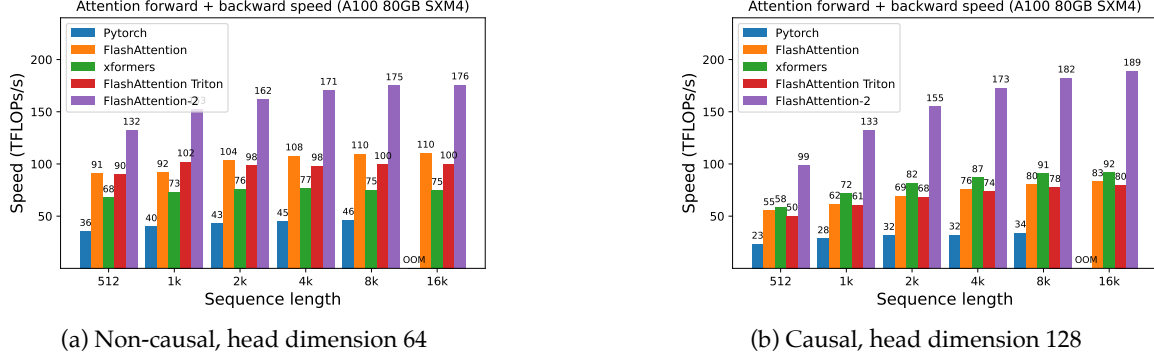


Figure 4.3: Forward and backward attention speed on A100 for representative FlashAttention-2 configurations. Source: [10].

The paper also evaluates H100. FlashAttention-2 benefits from faster hardware, but it does not fully exploit Hopper-specific asynchronous features. This motivates FlashAttention-3.

End-to-end training results demonstrate that the kernel improvements affect full Transformer workloads. The paper evaluates GPT-style models with 1.3B and 2.7B parameters on 8 A100 80GB SXM GPUs. For 8K context length, FlashAttention-2 provides a large improvement over both standard attention and FlashAttention. The model-scale context is close to Megatron-style large language model training, which the original paper cites as a relevant system baseline [40].

Table 4.2: End-to-end training results reported for FlashAttention-2

Model setting	Without FlashAttention	FlashAttention	FlashAttention-2
GPT3-1.3B, 2K context	142	189	196
GPT3-1.3B, 8K context	72	170	220
GPT3-2.7B, 2K context	149	189	205
GPT3-2.7B, 8K context	80	175	225

Values are TFLOPs/s/GPU as summarized from [10].

The evaluation supports the central claim of FlashAttention-2: after IO-aware tiling, the next layer of performance comes from execution structure. Sequence parallelism im-

proves occupancy, warp partitioning reduces communication, and algorithmic changes reduce scalar overhead.

4.4 Summary

FlashAttention-2 advances the series from IO awareness to parallelism awareness. It keeps the exact-attention and linear-memory benefits of FlashAttention, but reorganizes how work is distributed across thread blocks and warps. In operating-systems terms, it is a scheduling and synchronization optimization inside the GPU kernel. Its remaining gap is Hopper-specific: parallelism improvements transfer to H100, but they do not fully exploit asynchronous memory movement, Tensor Core work, and softmax overlap [28, 30]. That gap is the starting point for FlashAttention-3.

Chapter 5

FlashAttention-3: Asynchrony and Low-Precision Co-Design

5.1 Motivation

FlashAttention-3 is motivated by the transition from Ampere-style optimization to Hopper-specific optimization. FlashAttention-2 improves parallelism and work partitioning, but on H100 it still leaves a gap relative to the hardware’s peak Tensor Core capability [38]. The gap arises because Hopper introduces execution features that require a different schedule: asynchronous WGMMMA operations, TMA-based memory movement, and FP8 Tensor Core support [28–30].

The central observation is that attention contains heterogeneous work. GEMM uses Tensor Cores and can be extremely fast. Softmax and normalization involve reductions, exponentials, and scalar operations. Memory movement brings tiles from HBM to shared memory. If these stages execute serially, fast Tensor Core operations may wait for slower softmax or data movement. FlashAttention-3 therefore treats attention as a pipeline that must be overlapped.

Low precision introduces another motivation. FP8 can provide much higher throughput than FP16 or BF16, but attention is numerically sensitive because softmax can amplify outliers. The paper must therefore solve two problems at once: use FP8 hardware effectively and

control numerical error. This makes FlashAttention-3 a co-design paper across scheduling, precision, and hardware [24].

Table 5.1: Hopper features relevant to FlashAttention-3

Feature	Role in Hopper	Relevance to attention
WGMMMA	Asynchronous warpgroup matrix multiplication	Enables Tensor Core work to be issued while other work proceeds
TMA	Hardware-assisted tensor memory movement	Supports producer warps that move tiles efficiently
FP8 Tensor Cores	Higher low-precision throughput	Increases potential speed but requires numerical controls
Larger shared-memory pipeline	More opportunities for staging tiles	Makes producer-consumer scheduling more useful

5.2 Method

FlashAttention-3 introduces producer-consumer warp specialization. Within a cooperative thread array, some warpgroups act as producers that move data into shared memory using TMA, while other warpgroups act as consumers that perform WGMMMA and softmax-related computation. This separation allows the kernel to pipeline work more explicitly than a design in which all warps execute the same sequence of steps; warp specialization has precedent in GPU memory-bandwidth optimization [3].

The paper then introduces ping-pong scheduling and GEMM-softmax overlap. In a simple schedule, the kernel computes a GEMM, waits, performs softmax, and then computes the next GEMM. FlashAttention-3 overlaps these stages so that one warpgroup can execute softmax while another warpgroup issues the next WGMMMA. The goal is to reduce the idle time of Tensor Cores and hide the lower throughput of softmax operations.

The FP8 component requires additional design. FP8 arithmetic is faster but less precise. FlashAttention-3 uses block quantization and incoherent processing to reduce error under outlier-heavy input distributions. This is not a purely numerical change; it affects layout and data movement because FP8 Tensor Cores expect particular operand formats. The method therefore treats precision and kernel scheduling as coupled design choices, close to

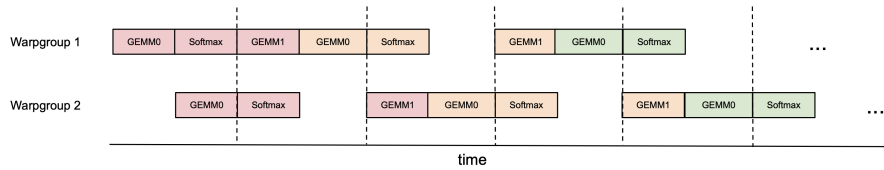


Figure 5.1: Ping-pong scheduling used in FlashAttention-3 to overlap softmax and matrix multiplication across warpgroups. Source: [38].



Figure 5.2: Pipeline designs in FlashAttention-3. The two-stage schedule overlaps WGMMMA and softmax, while the three-stage version further deepens the pipeline. Source: [38].

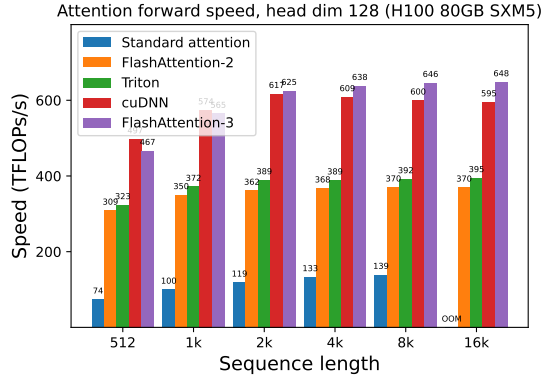
the broader CUTLASS-style view of exposing tiled GPU matrix operations to kernel developers [43].

Table 5.2: Method components in FlashAttention-3

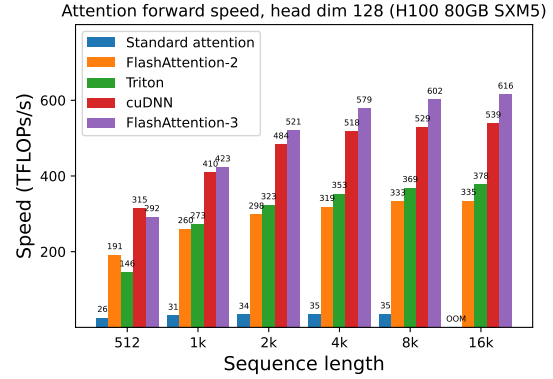
Component	Mechanism	Purpose
Warp specialization	Separate producer and consumer warpgroups	Overlap memory movement with compute
WGMMMA-softmax overlap	Execute softmax for one tile while issuing GEMM for another	Reduce Tensor Core idle time
Ping-pong scheduling	Alternate work between warpgroups	Maintain pipeline utilization
FP8-aware design	Use block quantization and incoherent processing	Improve throughput while controlling error

5.3 Experimental Evaluation

FlashAttention-3 reports substantial improvements on H100 GPUs. For FP16/BF16 forward propagation, the paper reports roughly $1.5\times$ to $2.0\times$ speedup over FlashAttention-2 and throughput up to about 740 TFLOPs/s [38]. For FP8 forward propagation, throughput approaches the PFLOPs/s regime. These results show the benefit of adapting the kernel to Hopper rather than relying only on the FlashAttention-2 schedule.

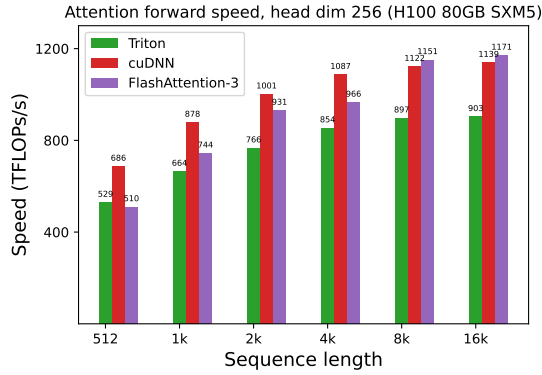


(a) Forward, non-causal, head dimension 128

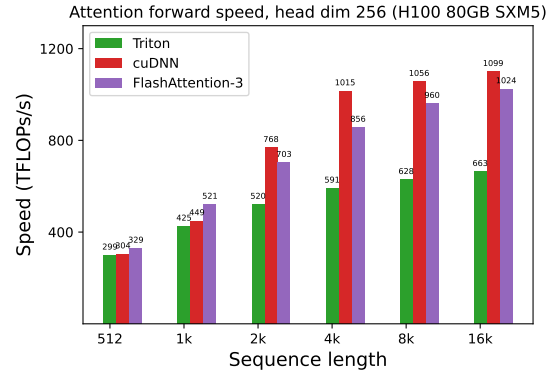


(b) Forward, causal, head dimension 128

Figure 5.3: FlashAttention-3 FP16/BF16 forward speed on H100 for representative causal and non-causal settings. Source: [38].



(a) FP8, non-causal, head dimension 256



(b) FP8, causal, head dimension 256

Figure 5.4: FlashAttention-3 FP8 forward speed on H100. FP8 increases throughput but requires explicit numerical design. Source: [38].

The ablation studies show that the gains come from the combined schedule. Removing GEMM-softmax pipelining or warp specialization reduces throughput. This supports the interpretation that FlashAttention-3 is not a collection of independent tricks. Its performance comes from coordinated overlap among memory movement, Tensor Core work, and softmax. This is consistent with the broader GPU-kernel lesson that work decomposition and scheduling can be as important as raw arithmetic throughput [31].

Table 5.3: Pipeline ablation results reported for FlashAttention-3

Configuration	Time (ms)	Throughput (TFLOPs/s)
FlashAttention-3	3.538	661
No GEMM-softmax pipelining, with warp specialization	4.021	582
With pipelining, no warp specialization	4.105	570

The numerical experiments are equally important. In FP16, FlashAttention-3 matches the accuracy behavior of FlashAttention-2 while improving speed. In FP8, the paper reports that block quantization and incoherent processing reduce error relative to a standard per-tensor FP8 baseline. This shows that low-precision performance cannot be evaluated only by throughput [24].

Table 5.4: Representative numerical error results reported for FlashAttention-3

Method	RMSE
Standard attention FP16	3.2×10^{-4}
FlashAttention-2 FP16	1.9×10^{-4}
FlashAttention-3 FP16	1.9×10^{-4}
Standard attention FP8 (per-tensor)	2.4×10^{-2}
FlashAttention-3 FP8	9.1×10^{-3}

5.4 Summary

FlashAttention-3 extends the series from parallelism-aware attention to hardware-specialized attention. It shows that newer GPU features can change the right execution schedule. Producer-consumer warp specialization, WGMMMA-softmax overlap, TMA-based data

movement, and FP8-aware numerical design together make attention more suitable for Hopper hardware.

From the perspective of this thesis, FlashAttention-3 completes a three-stage trajectory. FlashAttention reduces unnecessary HBM traffic. FlashAttention-2 improves occupancy and warp-level work assignment. FlashAttention-3 overlaps heterogeneous pipeline stages and incorporates low precision. The series therefore moves from memory locality, to scheduling, to hardware-specific asynchrony.

Chapter 6

Synthesis and Comparative Discussion

6.1 A Three-Stage Technical Trajectory

The three FlashAttention papers form a coherent technical trajectory. They all optimize exact attention, but each paper targets a different bottleneck: HBM traffic in FlashAttention, occupancy and work partitioning in FlashAttention-2, and Hopper-specific asynchronous execution in FlashAttention-3.

This is the systems pattern of bottleneck migration. Improving one resource often exposes pressure on another, just as a storage optimization can make CPU scheduling or synchronization overhead visible.

The trajectory also broadens what counts as the “algorithm”: first a blockwise recurrence, then a mapping to thread blocks and warps, and finally a hardware-aware pipeline and precision strategy.

6.2 Comparison by Systems Dimension

The three papers can be compared across several systems dimensions: locality, parallelism, synchronization, precision, and portability. FlashAttention makes locality the primary

Table 6.1: Bottleneck migration across the FlashAttention series

Paper	Bottleneck emphasized	Main response
FlashAttention	HBM traffic and quadratic intermediate storage	Tile attention, use online softmax, recompute in backward pass
FlashAttention-2	Insufficient occupancy and warp communication	Add sequence parallelism, reduce non-matmul operations, repartition work across warps
FlashAttention-3	Underuse of Hopper asynchronous and FP8 features	Use warp specialization, TMA, WGMMMA-softmax overlap, and FP8-aware numerical design

concern. FlashAttention-2 makes parallelism and synchronization the primary concern. FlashAttention-3 makes asynchrony and precision the primary concern. These dimensions are not independent, but each paper emphasizes a different one.

Table 6.2: Systems-level comparison of the three papers

Dimension	FlashAttention	FlashAttention-2	FlashAttention-3
Locality	Central contribution; avoids N^2 HBM intermediates	Preserved from FlashAttention	Preserved while adding Hopper pipeline
Parallelism	Uses blockwise execution but leaves utilization gap	Main contribution through sequence and warp partitioning	Uses specialized producer-consumer warpgroups
Synchronization	Reduces kernel boundaries through fusion	Reduces shared-memory communication among warps	Overlaps stages to avoid serial waits
Precision	Primarily FP16/BF16 exact attention	Primarily FP16/BF16 exact attention	Adds FP8 throughput and error control
Portability	Establishes general IO-aware substrate	Improves A100 and transfers to H100	Specialized to Hopper features

No single dimension explains the entire series. Each later paper preserves the previous foundation while exposing a new performance lens.

6.3 Experimental Patterns Across Papers

The experiments across the three papers combine kernel-level evidence with model-level relevance. FlashAttention reports runtime, memory, and training results; FlashAttention-2 reports kernel speed and GPT-style training throughput; FlashAttention-3 adds pipeline ablations and numerical-error measurements.

Second, the reported gains are strongest when the measured setting stresses the targeted bottleneck. FlashAttention is most valuable when sequence length is large enough that attention matrix materialization dominates. FlashAttention-2 is especially useful when batch size and head count do not provide enough parallel work. FlashAttention-3 is most valuable on Hopper GPUs where WGMMMA, TMA, and FP8 can be exploited. This means the correct interpretation of each paper depends on workload and hardware context.

Third, the experiments reveal different kinds of evidence. FlashAttention’s linear-memory behavior is structural evidence: it follows from the algorithmic dataflow and is visible in memory benchmarks. FlashAttention-2’s throughput improvement is execution evidence: it shows that better scheduling can improve hardware utilization. FlashAttention-3’s ablation and error studies are co-design evidence: they show that the speedup depends on overlap mechanisms and that low precision requires numerical safeguards.

Table 6.3: How the main experiments support each paper’s claim

Paper	Key experimental evidence	Claim supported
FlashAttention	Runtime and memory benchmarks; model training results	IO-aware exact attention improves both kernel and model-level performance
FlashAttention-2	A100/ H100 speed curves; GPT-style training throughput	Better parallelism and work partitioning improve utilization after IO reduction
FlashAttention-3	H100 FP16/ BF16 and FP8 speed; pipeline ablations; numerical error	Hopper-specific asynchronous and low-precision design is necessary for next-generation performance

6.4 Connection to Operating Systems Concepts

The FlashAttention series maps naturally to operating-systems concepts. The first concept is locality. FlashAttention manages the attention working set so that useful data remains in fast memory. This parallels cache-aware algorithms and virtual-memory working-set reasoning [12, 48]. The second concept is scheduling. FlashAttention-2 changes how work is distributed across blocks and warps so that resources are better utilized. This resembles CPU and thread scheduling, but at a much finer granularity.

The third concept is synchronization. FlashAttention-2 reduces unnecessary shared-memory communication, while FlashAttention-3 overlaps pipeline stages to reduce waiting. These are the same kinds of concerns that appear in multi-threaded operating systems: communication costs can erase parallelism if the schedule is poorly designed.

The fourth concept is hardware abstraction. Operating systems expose abstractions while still depending on hardware details for performance. FlashAttention-3 shows the tension clearly. A portable CUDA-level design may run on H100, but it may not use H100 well. Achieving high performance requires exposing lower-level hardware features such as TMA, WGMMMA, and FP8 layout [24, 29, 30]. This is similar to the tradeoff between portability and specialization in systems software.

6.5 Limitations and Open Questions

The three papers leave several open questions. One limitation is portability: FlashAttention-3 obtains its largest gains by specializing for Hopper, so future GPU generations may require another redesign. A long-term question is whether compiler systems can express such schedules at a higher level without losing performance [35, 37, 44].

Another limitation is distributed attention. Very long contexts often require multiple GPUs, KV-cache partitioning, or sequence parallelism across devices, where network communication and memory placement become first-order concerns [2, 39].

A third open question is low-precision training stability. FlashAttention-3 shows promis-

ing FP8 local error behavior, but future work must connect kernel-level error metrics with full training dynamics.

Finally, there is a programmability challenge: the best implementations still depend on deep CUDA and hardware knowledge, creating a gap between model researchers and kernel engineers.

6.6 Summary

The collective lesson is that exact attention remains viable for long-context Transformers when the algorithm is co-designed with the memory hierarchy, execution schedule, and numerical format.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis has reviewed FlashAttention, FlashAttention-2, and FlashAttention-3 as a connected sequence of systems optimizations for exact self-attention. The main conclusion is that the three papers should be read as a staged hardware-software co-design trajectory. Each paper preserves the mathematical definition of attention, but changes the execution strategy to address the bottleneck that is most important at its stage.

FlashAttention establishes the foundation by showing that exact attention does not need to materialize the full attention matrix in HBM. Its combination of tiling, online softmax, recomputation, and kernel fusion changes attention from a memory-intensive sequence of tensor operations into an IO-aware fused kernel. The core lesson is locality: performance improves when the working set is organized around fast memory.

FlashAttention-2 builds on that foundation by improving parallelism and work partitioning. Once HBM traffic is reduced, the remaining bottlenecks are occupancy, non-matmul overhead, and communication among warps. FlashAttention-2 therefore exposes more sequence-dimension parallelism and changes warp-level ownership of the output. The core lesson is scheduling: after locality is controlled, work must still be assigned to hardware resources efficiently.

FlashAttention-3 moves the trajectory to Hopper hardware. It uses producer-consumer

warp specialization, TMA, WGMMMA-softmax overlap, and FP8-aware numerical design. The core lesson is co-design: when hardware changes, the kernel schedule and numerical format must change with it. Hopper’s performance features are valuable only if the attention algorithm is reorganized to expose and overlap the right work.

Across the three papers, efficient exact attention evolves from IO awareness, to parallelism awareness, to hardware specialization. This progression is directly relevant to operating systems because it reflects the same principles that appear in memory management, scheduling, synchronization, and resource-aware performance analysis.

7.2 Future Work

Several directions naturally follow from this analysis. The first is distributed long-context attention. As context windows continue to grow, single-GPU optimization will not be enough. Future systems must coordinate FlashAttention-style kernels with sequence parallelism, tensor parallelism, pipeline parallelism, and KV-cache placement across multiple devices. In this setting, communication scheduling and interconnect bandwidth become as important as on-chip memory [2, 39].

The second direction is compiler support. Current high-performance attention kernels require low-level engineering and hardware-specific scheduling. Compilers and domain-specific languages may eventually express tiling, recomputation, warp specialization, and asynchronous pipelines more automatically. The challenge is to raise the abstraction level without hiding the hardware details that determine performance [35, 37, 44].

The third direction is low-precision reliability. FlashAttention-3 shows that FP8 attention can be both fast and reasonably accurate at the kernel level, but large-scale training stability remains a broader question. Future work should connect local numerical error, activation outliers, optimizer behavior, and long-horizon convergence. This is especially important as hardware trends push more computation into lower-precision formats [24].

The fourth direction is integration with model architectures. Exact attention optimization and alternative sequence models are often presented as competing directions, but they can

also be complementary. Long-context systems may use FlashAttention-style kernels for exact local or global attention while using state-space, retrieval, or sparse mechanisms elsewhere. Understanding how to compose these techniques is an open systems and modeling problem [4, 7, 15, 51].

7.3 Final Remarks

The FlashAttention series is valuable because it provides faster attention kernels, but its broader value is methodological. It shows how to analyze an AI operator as a systems problem: identify the real bottleneck, map the working set onto the memory hierarchy, schedule parallel work carefully, overlap heterogeneous stages, and adapt the design to hardware evolution. These lessons extend beyond attention and are likely to remain relevant as model workloads and accelerator architectures continue to change.

Appendix A

Review Protocol

This appendix keeps only the supporting material needed to make the literature review reproducible. The main text already contains the technical narrative; the appendix therefore records the selection criteria, comparison fields, and review boundaries rather than repeating the chapter-level discussion.

A.1 Paper Selection

The review centers on three papers: FlashAttention, FlashAttention-2, and FlashAttention-3. They were selected because they form a direct technical sequence. Each paper preserves exact scaled dot-product attention, but changes the implementation strategy to address a different systems bottleneck. This makes them suitable for a comparative study in an operating-systems course: the same mathematical operator is carried across changing memory, scheduling, and hardware constraints [10, 11, 38].

Related work was used for context rather than as an additional review target. Sparse attention, linear attention, state-space models, memory-hierarchy papers, and GPU architecture references help explain why the FlashAttention series matters, but they do not replace the three-paper comparison [1, 7, 15, 28, 46].

A.2 Extraction Fields

Field	Purpose in the review
Fixed operator	Confirms whether the method still computes exact attention.
Main bottleneck	Identifies whether the paper targets HBM traffic, occupancy, pipeline overlap, or numerical throughput.
Execution change	Records the concrete scheduling or dataflow modification.
Evidence type	Separates memory benchmarks, kernel throughput, end-to-end training results, ablations, and numerical-error results.
OS connection	Maps the method to locality, scheduling, synchronization, pipelining, or resource specialization.

Table A.1: Fields used to compare the three core papers.

A.3 Scope Boundaries

The thesis is a literature-based technical synthesis, not a full CUDA reproduction. It does not claim to reproduce the exact kernels or all reported numbers. The goal is to explain what changes from one paper to the next and why those changes are meaningful from a systems perspective.

The review also distinguishes algorithmic equivalence from implementation equivalence. A simple tiled attention implementation can preserve the same mathematical recurrence as FlashAttention, but still be much slower than the published kernel. The published results depend on exact attention, online normalization, careful tiling, and hardware-aware scheduling [10, 11, 25].

Appendix B

Supplementary Attention Derivation

B.1 Standard Attention

For one attention head, scaled dot-product attention is

$$O = \text{softmax}(QK^\top / \sqrt{d})V,$$

where Q , K , and V have N rows and head dimension d . A conventional implementation forms the score matrix $S = QK^\top / \sqrt{d}$, applies row-wise softmax to obtain P , and then computes $O = PV$. The matrices S and P each have N^2 entries, so storing them creates quadratic memory pressure [33, 45].

B.2 Online Softmax

For a row vector x , stable softmax uses

$$\text{softmax}(x)_j = \frac{\exp(x_j - m)}{\sum_k \exp(x_k - m)}, \quad m = \max_k x_k.$$

When the row is processed block by block, the maximum and denominator can be updated online. Let the old prefix have maximum m_{old} and denominator ℓ_{old} . Let the new block have

maximum m_{blk} and denominator ℓ_{blk} . The combined maximum is

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{blk}}),$$

and the combined denominator is

$$\ell_{\text{new}} = e^{m_{\text{old}} - m_{\text{new}}} \ell_{\text{old}} + e^{m_{\text{blk}} - m_{\text{new}}} \ell_{\text{blk}}.$$

The output accumulator is rescaled in the same way. This compact state is what allows each local score tile to be consumed and discarded without writing the full attention matrix to HBM [11, 25].

B.3 IO Accounting

Object	Conventional attention	FlashAttention-style dataflow
Q, K, V	Stored in HBM and read by separate kernels.	Streamed through tiles and reused on chip where possible.
$S = QK^\top$	Materialized or exposed as a full score matrix.	Exists only as a local tile.
$P = \text{softmax}(S)$	Materialized or saved for backward.	Folded into row statistics and output accumulators.
Softmax statistics	Implicit in a full softmax pass.	Explicit per-row state carried across tiles.
Backward state	Often saves large attention intermediates.	Saves compact statistics and re-computes local tiles.

Table B.1: Simplified lifetime comparison for exact attention.

The key tradeoff is memory for recomputation. FlashAttention reduces stored activation state and HBM traffic, while the backward pass recomputes local attention quantities from saved row statistics. This is a standard systems tradeoff: when memory traffic is more ex-

pensive than extra arithmetic, recomputation can improve overall performance [6, 11].

Appendix C

Reproducibility Checklist

C.1 Benchmark Settings

A reproduction-oriented study should vary the settings that control both attention structure and GPU utilization. Reporting a single speedup number is not enough, because the three papers optimize different bottlenecks.

Setting	Why it matters
Sequence length	Controls the quadratic attention structure and the size of avoided intermediates.
Head dimension	Changes tensor-core efficiency, register pressure, and tile shape.
Batch size and number of heads	Determine how much natural parallelism exists outside the sequence dimension.
Causal mask	Changes control flow and the amount of useful work.
Forward vs. backward	Tests whether recomputation and saved statistics are handled efficiently.
Precision format	Affects memory traffic, numerical error, and tensor-core use.
GPU generation	Determines whether A100-style or Hopper-specific mechanisms are available.

Table C.1: Benchmark dimensions for a focused reproduction study.

C.2 Correctness Checks

Correctness should be checked before performance. The optimized kernel should be compared with a high-precision reference implementation for small, medium, and long sequence lengths. Both causal and non-causal cases should be tested. For training, gradients with respect to Q , K , and V should be compared against the reference implementation.

The tolerance must match the precision format. FP16, BF16, and FP8 have different expected error profiles, so the report should state the metric used for numerical comparison. Maximum absolute error, relative error, and end-to-end model quality answer different questions.

C.3 Performance Reporting

Performance measurements should include warm-up iterations, repeated trials, and a clear baseline. The report should state the GPU model, CUDA version, framework version, precision, sequence length, batch size, head count, and whether the measurement includes forward only or forward plus backward [29, 32].

End-to-end measurements are useful when available. A faster attention kernel may produce a smaller full-model speedup if attention is not the dominant component. Kernel-level results explain the mechanism; model-level results explain the practical effect.

Appendix D

Supplementary Systems Mapping

This final appendix records the compact systems mapping used by the thesis. It is intentionally brief: the detailed argument belongs in Chapter 6, while this appendix only lists the supporting associations and limitations.

D.1 Operating-Systems Concepts

Concept	FlashAttention interpretation	Main paper
Locality	Keep active tiles and row statistics near compute; avoid large HBM intermediates.	FlashAttention
Scheduling	Expose sequence-level and warp-level work after IO traffic is reduced.	FlashAttention-2
Synchronization	Reduce cross-warp communication and unnecessary barriers.	FlashAttention-2
Pipelining	Overlap data movement, tensor operations, and softmax work.	FlashAttention-3
Specialization	Use Hopper mechanisms and FP8-aware design when worthwhile.	FlashAttention-3

Table D.1: Compact mapping from FlashAttention techniques to OS concepts.

D.2 Limitations

Three limitations are most relevant to the thesis argument. First, portability decreases as the series moves from FlashAttention to FlashAttention-3. The IO-aware idea is broad, but

Hopper-specific scheduling is less portable. Second, numerical behavior becomes more important as lower precision is used. Exact attention and low-precision execution are related but separate claims. Third, benchmark scope matters. A result for one sequence length, head dimension, and GPU does not establish a universal ranking [24, 38].

These limitations do not weaken the main conclusion. They clarify it: the FlashAttention series is best understood as a sequence of systems optimizations whose value depends on workload, hardware, and the level of the software stack being measured.

Bibliography

- [1] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [3] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU memory bandwidth via warp specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, SC '11*, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400.
- [4] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [5] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- [7] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [8] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.
- [9] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.
- [10] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint*, 2023.

- [11] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [12] Peter J Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019.
- [14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.
- [15] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The International Conference on Learning Representations (ICLR)*, 2022.
- [16] John Hennessy and David Patterson. Memory hierarchy design. *Computer Architecture: A Quantitative Approach*, pages 390–525, 2003.
- [17] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems*, 3:711–732, 2021.
- [18] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.
- [19] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.
- [20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.
- [21] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.
- [22] Benjamin Lefaudeux, Francisco Massa, Diana Liskovich, Wenhan Xiong, Vittorio Caggiano, Sean Naren, Min Xu, Jieru Hu, Marta Tintore, Susan Zhang, Patrick Labatut, and Daniel Haziza. xformers: A modular and hackable transformer modelling library. <https://github.com/facebookresearch/xformers>, 2022.
- [23] Frank McSherry, Michael Isard, and Derek G Murray. Scalability! but at what {COST}? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*, 2015.

- [24] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.
- [25] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.
- [26] NVIDIA. Nvidia Tesla V100 GPU architecture, 2017.
- [27] NVIDIA. Nvidia A100 tensor core GPU architecture, 2020.
- [28] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.
- [29] NVIDIA. CUDA programming guide version 12.4, 2024. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [30] NVIDIA. Parallel thread execution ISA version 8.4, 2024. URL https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf.
- [31] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-K: Work-centric parallel decomposition for dense matrix-matrix multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPOPP '23*, pages 429–431, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700156. doi: 10.1145/3572848.3577479.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- [33] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.
- [34] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [35] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6): 519–530, 2013.
- [36] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021.
- [37] Amit Sabne. XLA: Compiling machine learning for peak performance, 2020.
- [38] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint*, 2024.

- [39] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [40] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [41] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2020.
- [42] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.
- [43] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, jan 2023. URL <https://github.com/NVIDIA/cutlass>.
- [44] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [45] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [46] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- [47] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [48] Michael E Wolf and Monica S Lam. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 30–44, 1991.
- [49] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.

- [50] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nystöm-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.
- [51] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.